

Dewy AI 웨딩플래너

DB-Grounded RAG & 환각 방지 파이프라인

기술 리뷰어용 · ai-planner 엣지평션 + 챗봇 핸들러 walkthrough

근거: docs/260612_ai_planner_caching_grounding_plan.md (L1~L5 설계 권위) · 본 자료는 현재 코드 기준

TL;DR — 우리가 쓰는 RAG의 정확한 정체

- [O] Retrieval-Augmented Generation 맞음 — LLM이 답하기 전 앱 내 실데이터를 프롬프트 근거로 주입하고, 그 밖의 수치·업체명 생성을 금지한다.
- [X] 벡터/임베딩 RAG 아님 — pgvector ·embedding 컬럼·코사인 유사도·<=> 연산자·HNSW/IVFFlat 없음(마이그레이션 grep 0건).
- 정확한 라벨: *Structured/keyword retrieval* → *prompt augmentation* → *guarded generation* → *output audit*.
- 도메인 특성상 이 선택이 합리적: 업체 카탈로그는 **정형 데이터**(지역·카테고리·가격·평점)라 **정확 필터**가 의미 검색보다 정밀하고, 환각 1순위 위험(없는 업체·가격 날조)을 **하드 제약**으로 막는 게 핵심.

팀 문서가 이 레이어를 명시적으로 "**L2. 근거 주입 (RAG)**" 라 부른다.

문제 정의 — 왜 그냥 LLM이면 안 되나

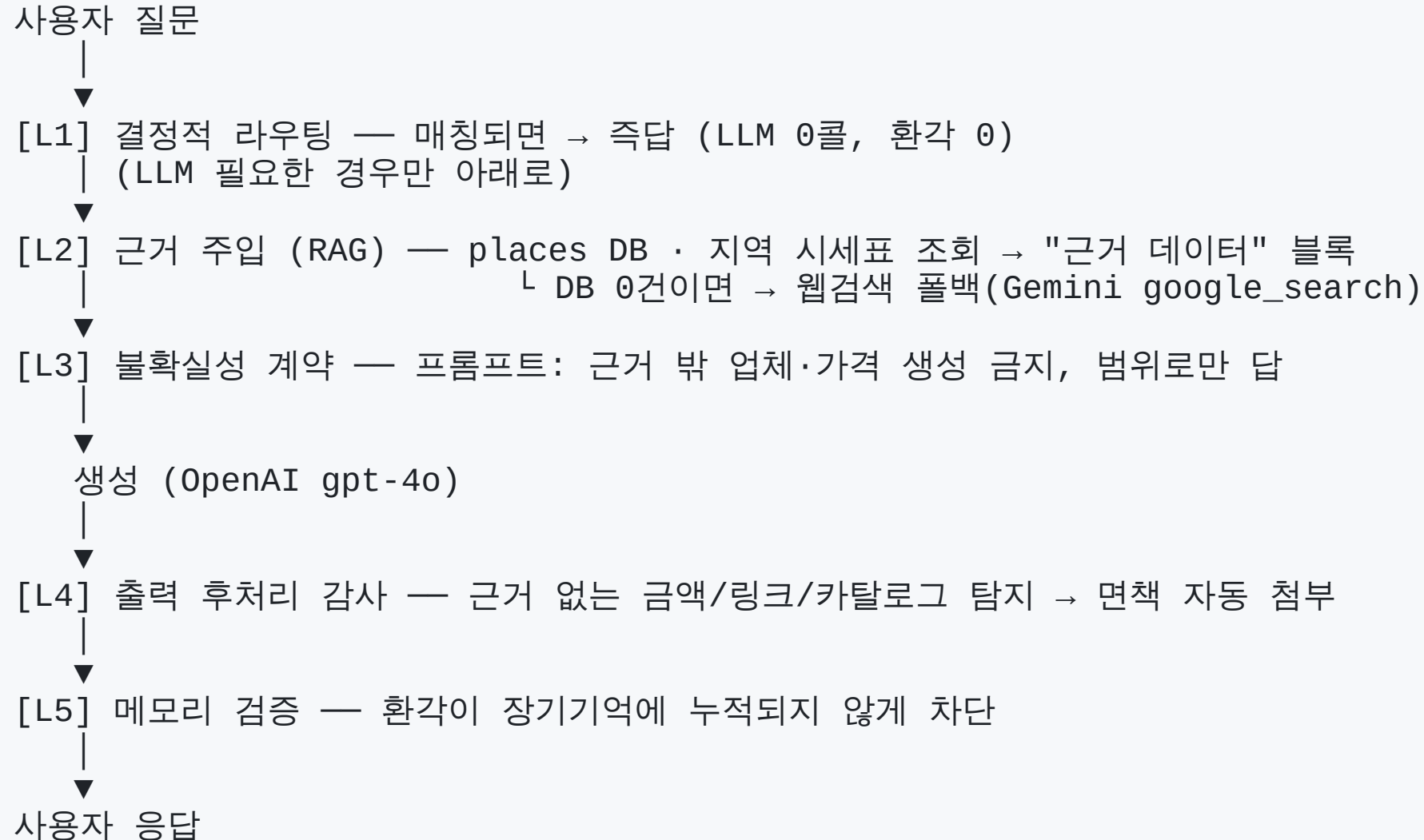
웨딩 도메인에서 가장 치명적인 환각:

1. 없는 업체명을 지어냄 ("○○웨딩홀 추천!" → 실재 X) → 사용자 신뢰 즉사
2. 가격을 단정 ("강남 웨딩홀 식대 7만원" → 시즌·홀별 천차만별)
3. 가짜 링크/전화번호 생성

→ 일반 챗봇 UX로는 "그렇듯하지만 틀린" 답이 **에러 없이** 나간다.

→ 대응: **5단계 방어 계층(L1~L5)**. RAG(L2)는 그 중 retrieval 단계.

아키텍처 — 5단계 방어 계층



파일 맵 (retrieval → augmentation → generation → audit)

단계	파일	핵심
L1 결정 적	<code>src/lib/chatbot/intentRouter.ts</code>	의도 분류, 핸들러 즉답
L1 핸들 러	<code>src/lib/chatbot/handlers/searchHandlers.ts</code>	카테고리·지역 키 워드 → places 조 회
L2 검색 → 근 거	<code>supabase/functions/ai-planner/grounding.ts</code>	가격·업체 근거 블 록 빌드

L1 — 결정적 우선 (가장 강한 환각 방지)

`src/lib/chatbot/intentRouter.ts` + `handlers/`

- LLM 호출 전 의도를 분류. 가격·업체검색·평균 등 정형 질문은 **핸들러가 DB만으로 즉답**.
- 매칭되면 LLM 0콜 → 환각 0, 지연 ↓, 비용 ↓.
- 예: `handleAveragePrice`, `handlePopularPlaces`, `handleVenueCompare` (업체 3~10곳 표).

RAG의 전제: "LLM을 최대한 안 부른다." 부를 때만 L2가 근거를 채운다.

L2 — 가격 근거 (Retrieval ①)

supabase/functions/ai-planner/grounding.ts:63-130

```
export function isPriceQuery(text: string): boolean    // 평균·얼마·시세·비용·예산...
export function buildPriceGrounding(text, region, guestCount): string
```

- 트리거: 가격 키워드 + 웨딩 항목(웨딩홀·식대·메이크업·예복·한복·예물·허니문) 정
규식.
- 검색 소스: **16개 지역 × 카테고리 시세표**(구조화 상수).
- 출력: "근거 데이터 (가격)" 블록 → 시스템 프롬프트로.
- 프롬프트 규칙: *이 표의 숫자만 인용, 없으면 "데이터 없음".*

L2 — 업체 근거 (Retrieval ②)

grounding.ts:192-220 — buildVendorGrounding()

```
await supabase.from("places")
  .select("name, category, city, district, avg_rating, min_price, is_partner")
  .eq("is_active", true).eq("category", cat)
  .or(`district.ilike.%${sub}%,city.ilike.%${sub}%`) // 고정 substring만
  .order("is_partner", { ascending: false })
  .order("avg_rating", { ascending: false, nullsFirst: false })
  .limit(5);
```

- top-5 실제 업체 → "근거 데이터 (업체)" 블록. 제약: 목록 밖 업체명 생성 금지.
- 보안: 원문 사용자 입력을 ILIKE/or() 에 직접 넣지 않음 — 사전 정의된 고정 substring만(인젝션 방어, grounding.ts:203).
- 회귀 방지: 지역 약자 → 연속 substring 매핑(REGION_TO_ILIKE). "충남"을 그대로 ILIKE하면 "충청남도" 비연속 매칭 실패로 0건 되던 버그 차단.

L2 — 웹검색 폴백 (DB 0건일 때)

webSearchFallback.ts · supabase/functions/vendor-web-search/index.ts

```
if (db.length === 0) {  
  const web = await callWebSearch("search", userMessage, { category, region });  
}
```

- DB 근거가 비면 **Gemini 2.5 Flash + Google Search Grounding** 호출.
- 출처 검증: 응답을 그대로 믿지 않고
groundingMetadata.groundingChunks[].web.uri 의 검증된 **URI**만 추출(LLM이 URL 날조 못 하게).
- 즉, 폴백조차 "모델이 생성한 사실"이 아니라 "검색 엔진이 반환한 근거"에 고정.

증강 (Augmentation) + 프롬프트 캐싱

supabase/functions/ai-planner/index.ts:237-238

```
const dynamicContext = conditionalCapsules + userContext
                        + priceGrounding + vendorGrounding.block;
const systemPrompt = staticPrompt + dynamicContext;
```

- **static / dynamic 분리**: 변하지 않는 시스템 지침은 앞(캐시 적중), 질문별 근거는 뒤 (`dynamicContext`).
- OpenAI automatic prompt caching으로 정적부 재사용 → 비용·지연 절감(Part A).
- 근거 블록은 이 호출에서 검색된 실데이터만 포함.

L3 — 불확실성 계약 (생성 제약)

supabase/functions/ai-planner/prompt.ts:63-70

수치·사실 신뢰성 계약 (환각 방지):

- 구체적 금액·통계·비율은 항상 '범위'로(단일 단정 금지)
- "지역·시즌·업체별 상이" 단서와 함께
- **특정 업체명·상품명·가격을 지어내지 마세요(가장 흔한 치명적 오류)**
- 근거 없으면 범위만 제시하거나 "견적받아보세요"로 회피

- 생성 모델: **OpenAI gpt-4o** (index.ts:19 , CHAT_MODEL).
- 비용 레이어(Part A3): gpt-4o-mini 기본 / gpt-4o 승격 2-tier 라우팅.

L4 — 출력 후처리 감사 (값싼 휴리스틱)

supabase/functions/ai-planner/postprocess.ts:42-66 — auditFullText()

```
// 근거 없이 금액이 나왔는데 헤지(범위/상이) 표현이 없으면 → 가격 면책 첨부
if (!ctx.hasPriceGrounding && AMOUNT_RE.test(text) && !HEDGE_RE.test(text))
  appendix = PRICE_DISCLAIMER;
// 근거 없이 가짜 링크 / 업체 2곳+ 카탈로그 / 미근거 업체의도 → 업체 면책 첨부
if (!hasGrounding && (fabricatedLink || catalogCount >= 2 || ungroundedVendorIntent))
  appendix += VENDOR_DISCLAIMER;
```

- 생성된 전체 응답을 다시 스캔: 근거 없는 금액·날조 링크·업체 나열 탐지.
- 발견 시 사용자에게 보내기 전에 면책 문구를 자동으로 덧붙임.
- SSE 스트림에도 적용(createSseAuditTransform).

End-to-End 예시 — "강남 웨딩홀 추천해줘"

질문: "강남 웨딩홀 추천해줘, 식대도 궁금"

- L1 intentRouter → 업체검색+가격 의도 (단, 자유서술이라 LLM 경로)
- L2 buildVendorGrounding("wedding_hall", "강남")
 - places 5곳(파트너·평점순) → "근거(업체)" 블록
 - buildPriceGrounding → 강남 웨딩홀 시세표 → "근거(가격)" 블록
- 증강: systemPrompt = static + (업체블록 + 가격블록)
- L3 계약: "이 목록 밖 업체명 금지, 가격은 범위로"
- 생성(gpt-4o): 5곳 실제 업체 + 범위가격 답변
- L4 감사: 근거 있음 → 면책 불필요(근거 없었으면 면책 첨부)

- 핵심: 답변의 업체·가격은 전부 L2에서 검색된 실데이터. 모델은 문장만 짓는다.
- DB에 강남 웨딩홀이 0건이면 → 웹검색 폴백 → 그래도 없으면 "데이터 없음".

L5 — 메모리 검증

`supabase/functions/ai-planner/memory.ts`

- 대화에서 추출한 사실을 장기기억에 쓰기 전 검증 → 환각이 기억에 누적되어 이후 답을 오염시키는 것을 차단.
- "한 번의 환각이 영구 진실이 되는" 실패 모드 방지.

왜 벡터 RAG가 아닌가? (의도된 트레이드오프)

측면	Dewy(현재)	벡터 RAG
검색 키	카테고리·지역·가격·평점 정확 필터	의미 유사도
인프라	추가 0(기존 <code>places</code> 재사용)	pgvector·임베딩 파이프라인·재색인
데이터 성격	정형 카탈로그	비정형 문서
환각 가드	하드 제약 + 출력 감사	근거 청크

- 업체/시세는 정형이라 **ILIKE + 정렬 + LIMIT**이 의미검색보다 **정밀·저렴·설명가능**.
- **벡터가 빛날 지점(향후)**: 후기·꿀팁 영상·커뮤니티 글 같은 **비정형 텍스트** 시맨틱 검색.

검증 & 로드맵

현재 검증 포인트

- L1 즉답 핸들러: DB 쿼리 단위 테스트 + 회귀(지역 ILIKE 0건) 케이스.
- L4 감사: `aiPlannerPostprocess` 테스트(근거 무/유 시 면책 분기).
- 인젝션: 사용자 원문 미투입(고정 substring) 정적 확인.

벡터 RAG로 확장한다면(향후)

1. `pgvector` + 후기/꿀팁 임베딩 테이블 → `match_*` RPC.
2. L2에 하이브리드 **retrieval**(구조 필터 n 시맨틱) 추가.
3. L3·L4 가드는 그대로 재사용(아키텍처가 retrieval 소스와 분리돼 있어 교체 용이).

요약

- Dewy는 **DB 근거주입형 RAG** — retrieval(L2) → augmentation → **가드된 생성(L3)** → 출력 감사(L4).
- 벡터 RAG는 아니지만, 도메인(정형 카탈로그 + 환각 치명)에선 **더 정밀·저렴·설명 가능한 선택**.
- 핵심 자산: **검색 소스와 가드의 분리** → 나중에 벡터 retrieval을 끼워도 L3/L4 재사용.

코드 출발점: `supabase/functions/ai-planner/grounding.ts` → `index.ts` → `postprocess.ts`